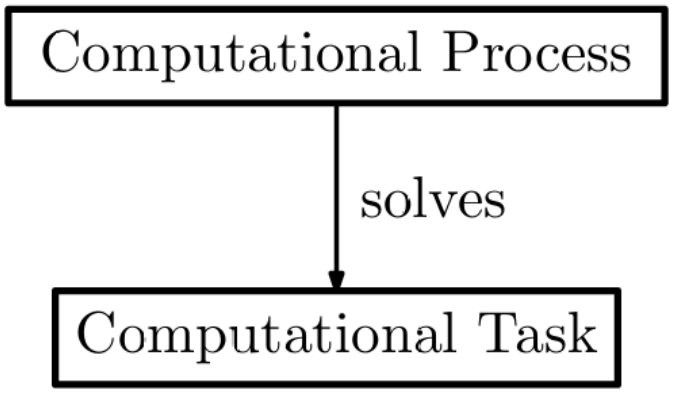
01_Matematical_Preliminaries

Wednesday, March 25, 2026 4:17 PM

Major Concepts and Notation



What *is* a Process?

- In the Theory of Computation, a process is taken to be an **algorithm**.
- Giving a precise definition of an algorithm is not possible *at this point*, but any algorithm must satisfy the following constraints:
  - It must be a *finite* description of a series of computation *steps*
  - Each step must be *elementary*.
  - The way the next step is determined must be *deterministic*

Sets and Numbers

- The cardinality of any set X is indicated as $|X|$.
  - It can be finite or infinite.
- We will mainly work with discrete numbers.
  - $\mathbb{N}$ is the set of natural numbers, while $\mathbb{Z}$ is the set of integers.

The phrase "**sufficiently large** n" is a formal mathematical way to describe a condition that becomes

permanently true once a certain threshold is passed. It means there exists a specific natural number N such that the property $P(n)$ is guaranteed to hold for every n that is greater than N.
This concept is essential in the theory of computation for several reasons:

- **Ignoring Finite Exceptions:** It allows mathematicians and computer scientists to focus on the **asymptotic behavior** of a function or algorithm, effectively ignoring a finite number of "small" cases where the condition might not yet be true.
- **Asymptotic Notation (Big O and Ω):** The definitions of $O(g)$ and $\Omega(g)$ rely entirely on this concept. For example, a function $f(n)$ is $O(g(n))$ if there is a constant c such that $f(n) \leq c \cdot g(n)$ for **sufficiently large** n. This means the upper bound only needs to hold after the input size reaches a certain point.
- **Robustness of Complexity Classes:** In defining the **class P**, the sources note that for any polynomial P, there are constants $c, d > 0$ such that $P(n) \leq c \cdot n^d$ for sufficiently large n. This ensures that the classification of a task as "efficiently solvable" depends on its long-term growth rate rather than its performance on small, specific inputs.

- For a given real number x, $\lceil x \rceil$ is the the smallest element of $\mathbb{Z}$ such that $\lceil x \rceil \geq x$. Whenever a real number x is use in place of anatural number, we implicitly read it as $\lceil x \rceil$

The notation $\lceil x \rceil$ (known as the ceiling function) represents the **smallest integer** (Z) that is **greater than or equal to** the real number x.
In the theory of computation, this convention is essential because parameters such as the number of computational steps or the size of an input must be **natural numbers**. Therefore, whenever a real number x (resulting from formulas like logarithms or polynomials) is used where a natural number is required, it is **implicitly interpreted as** $\lceil x \rceil$ to ensure the value is a valid discrete integer.

- For a natural number n, $[n]$ is the set $\{1, \ldots, n\}$.
- Contrarily to the common mathematical notation, $\log x$ is **base** 2 logarithm.

Strings

- If S is a finite set, then a **string** over the alphabet S is a finite, ordered, possibly empty, tuple of elements from S.
  - Most often, the alphabet S will be the set $\{0, 1\}$.
- The set of all strings over S of length exactly $n \in \mathbb{N}$ is indicated as S^n (where S^0 is the set containing only the empty string ε).
- The set of *all strings* over S is $\bigcup_{n=0}^{\infty} S^n$ and is indicated as S^*.
- The concatenation of two strings x and y over S is indicated as xy or as $x \cdot y$. The string over S obtained by concatenating x with itself $k \in \mathbb{N}$ times is indicated as x^k.
  - Strings form a monoid!
- The **length** of a string x is indicated as $|x|$.
- Examples

$$\{0, 1\}^2 = \{00, 01, 10, 11\} \qquad |000101| = 6$$

Functions

- Given two sets X and Y, their **cartesian product** $X \times Y$ is the set of all pairs of objects, the first one from X and the second one from Y:

$$X \times Y = \{(x, y) \mid x \in X \text{ and } y \in Y\}$$

- A **relation** on X and Y is a subset of $X \times Y$.
- A **function** f from X to Y is a relation on X and Y such that for every $x \in X$ there exists *exactly one* $y \in Y$ such that $(x, y) \in f$. We write in this case that $f : X \to Y$.

Tasks as Functions

- One of the principles of computational complexity is that any task of interest consists in *computing a function* from $\{0,1\}^*$ to itself.
- If the input and/or the output are not strings, but they are taken from a discrete set, they can be **represented** as strings, following some encoding, which however should be as simple as possible.

- Summing up, we always assume that the task we want to solve **is given** as a function $f : A \to B$ where both the domain A and B are discrete (i.e. countable) sets, sometime leaving the encoding of A and B into $\{0,1\}^*$ implicit.
- The encoding of any element x of A as a string is often indicated as $\llcorner x \lrcorner$ or simply as x.

Representing Objects as Strings: Examples

- Strings in S^*, where $S = \{a, b, c\}$.
  - a can be encoded as 00, b becomes 01 and c becomes 10.
  - this way, e.g., $abbc$ becomes 00010110.
- Natural numbers.
  - We can take of the many encodings of natural numbers as strings.
  - As an example, 12 becomes 1100.
- Pairs of binary strings, i.e. $\{0,1\}^* \times \{0,1\}^*$.
  - One can encode the pair (x, y) as the string $x\#y$ over the alphabet $\{0, 1, \#\}$, and the proceed as before.

Languages: Boolean functions

An important class of functions from $\{0,1\}^*$ to $\{0,1\}^*$ are those whose range are strings of length exactly one, called **boolean functions**.

- They are the so -called *characteristic* functions.

All functions that output 1 or 0 can be seen as classification function of two classes (0 and 1)
For that reason they are called classification functions

We identify such a function f with the subset $\mathcal{L}_f$ of $\{0,1\}^*$ defined as follows:

$$\mathcal{L}_f = \{x \in \{0,1\}^* \mid f(x) = 1\}.$$

A classification function can be defined as the set of all the value that given in input to the function The function will return 1.

Example:
A function that return 1 if x is a prime number can be represented by the set of all the Prime number

- Any subset of S^* (the set of all strings over an alphabet S) is usually called a **language**.
- This way, a **decision problem** for a given language $\mathcal{M}$ (i.e. does $x \in \{0,1\}^*$ is in $\mathcal{M}$?) can be seen as the task of computing f such that $\mathcal{M} = \mathcal{L}_f$.

Asymptotic Notation

- A function $f : \mathbb{N} \to \mathbb{N}$ is $O(g)$ if there is a positive real constants c such that $f(n) \leq c \cdot g(n)$ for sufficiently large n.
  - Example: the function $n \mapsto 3 \cdot n^2 + 4 \cdot n$ is $O(n^2)$, but also $O(n^3)$, and certainly $O(2^n)$. It is not, however, $O(n)$.
- A function $f : \mathbb{N} \to \mathbb{N}$ is $\Omega(g)$ if there if a positive real constants c such that $f(n) \geq c \cdot g(n)$ for sufficiently large n
  - Example: the function $n \mapsto 3 \cdot n^2 + 4 \cdot n$ is $\Omega(n^2)$, but also $\Omega(n)$, but not $\Omega(n^3)$.
- A function f is $\Theta(g)$ if f is both $O(g)$ and $\Omega(g)$.
  - Example: the function $n \mapsto 3 \cdot n^2 + 4 \cdot n + 7$ is $\Theta(n^2)$.

ABOUT CARDINALITIES

- $|S^n| = |S|^n$

The cardinality of the set of all strings of length n is equal to the cardinality of the **alphabet raised** to the power of n

SET OF FUNCTIONS WITH DOMAIN X AND CODOMAIN Y

- $|Y^X| = |Y|^{|X|}$

Exercise about comparing the asymptoric relation of different function.
To compare different function we use do the limit of the fraction between the two function

EXERCISE

RELATE THE FOLLOWING PAIRS OF FUNCTIONS BY WAY OF THE ASYMPTOTIC OPERATORS

$f_1(n) = n \lg n$ $\quad g_1(n) = 10\, n \lg(\lg n)$

$f_2(n) = 1000\, n$ $\quad g_2(n) = \frac{1}{100} n \lg n$

SOLUTION

$$\lim_{n\to\infty} \frac{f_2(n)}{g_2(n)} = \lim_{n\to\infty} \frac{\cancel{x} \lg n}{10\cancel{x}\lg(\lg n)} = +\infty$$

Exercise About Converting things to string

This is useful because our definition of functions use strings as inputs

1) Exercise to represent the numbers in Q as strings

(A) WE CAN OBSERVE THAT

$\mathbb{Q} = \{ z/n \mid z \in \mathbb{Z}, n \in \mathbb{N}, n > 0 \}$

SINCE ELEMENTS OF $\mathbb{Q}$ ARE FRACTIONS, WE CAN FIRST REDUCE THEM IN SUCH A WAY THAT z, n DO NOT HAVE ANY NONTRIVIAL FACTOR IN COMMON, AND THEN ENCODE THE RESULTING PAIR OF NUMBERS AS USUAL, USING A SEPARATOR:

$\llbracket z/n \rrbracket = \llbracket s_z \; m_z \; \# \; n \rrbracket$

m_z → ABSOLUTE VALUE OF z

s_z → SIGN OF z

$\llbracket -2/3 \rrbracket = \llbracket 1\,1\,0 \; \# \; 1\,1 \rrbracket = 0\,1\,0\,1\,0\,0\,1\,0\,0\,1\,0\,1$

$1\;1\;0\;\#\;1\;1$

2) Exercise to represent the disjoint union of the set of natural number (X) And the set of strings (Y)

what is a disjoint union?

The formal definition of a **disjoint union** ($X \uplus Y$) is a way to combine two sets while keeping their elements distinct, even if the sets share the same values. By attaching a **label** or "marker" (l for left/first set and r for right/second set) to every element, you ensure that you can always trace an element back to its original set.

Imagine two sets that overlap:

- **Set X** = {1, 2}
- **Set Y** = {2, 3}
  Standard Union ($X \cup Y$): {1, 2, 3}. In a normal union, the number '2' appears only once, and you lose the information of whether it came from X, Y, or both.
  Disjoint Union ($X \uplus Y$): Instead of just taking the numbers, we take **pairs**:
  - From X, we get: **(1, l)** and **(2, l)**
  - From Y, we get: **(2, r)** and **(3, r)**
  - **Result:** {(1, l), (2, l), (2, r), (3, r)}.

Because of the labels, (2, l) and (2, r) are treated as **different elements**. This allows the "new" combined set to preserve the identity of the source set for every single member.

Solution of the exercise

this concept is used to create **encodings** for different types of data into a single format (like binary strings).

If we want to create a disjoint union of **Natural Numbers (N)** and **Binary Strings ($\{0, 1\}^*$)**, we use bits as the labels:

- **Label l (left)** becomes the bit **0**.
- **Label r (right)** becomes the bit **1**.
- **Example 1:** To represent the number **5** (from the left set N, we take its binary form 101 and attach the label 0. The result is **0101**.
- **Example 2:** To represent the binary string **101** (from the right set $\{0,1\}^*$), we attach the label 1. The result is **1101**.

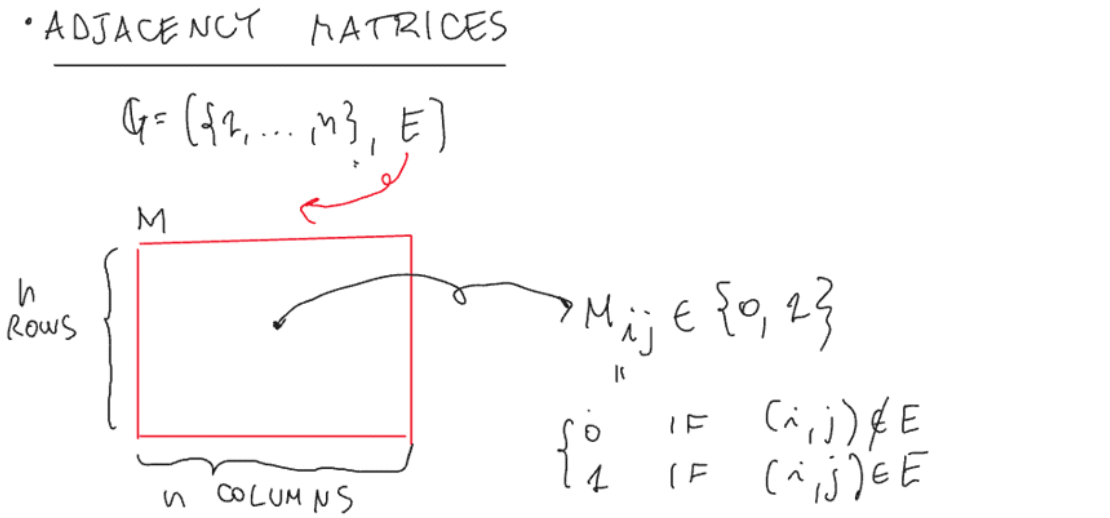
Even though both original elements "looked" like 101, their encoded versions (0101 vs 1101) are distinct because the disjoint union identifies exactly which set they belong to.

$\llcorner (n,e) \lrcorner = 0 \llcorner n \lrcorner$
$\llcorner (s,r) \lrcorner = 1\ s$

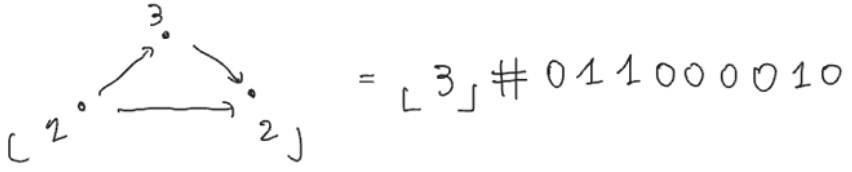
3) Represent a Graph a string

First Method:
Basically we create a matrix M with n columns and n rows, where n is the number of nodes. Each element M_{ij} of the matrix symbolize If there is an edge between the node i and the node j. Because we need to create a string the matrix must be flatten in a single row. To distinguish the row i from the row $i+1$, we need to encode also the row length n at the start of the matrix (we also separate this number with a separator)

This method require to build a matrix n^2, so it requires quite a lot fo space that is wasted for sparce matrices. But it optimize the encoding of the connections for dense matrices



$\llcorner G \lrcorner = \llcorner n \lrcorner \# M_{11} M_{12} \cdot M_{1n} M_{21} \cdots \cdot M_{nn}$



Second method:
With this second method, for each node we encode the list of every node directly connected to it.

It's important to notice that this method require more symbol to encode a single edge, because it has to write the number of the node. While for the first method every single edge is encode as a single bit.

For that reason this method is better for sparse matrices

ADJACENCY LISTS

- WHEN THE NUMBER OF EDGES IS ASYMPTOTICALLY SMALLER THAN $|V|^2$, ONE COULD RATHER REPRESENT E AS A LIST OF LISTS

$V = \{1, 2, 3, 4\}$

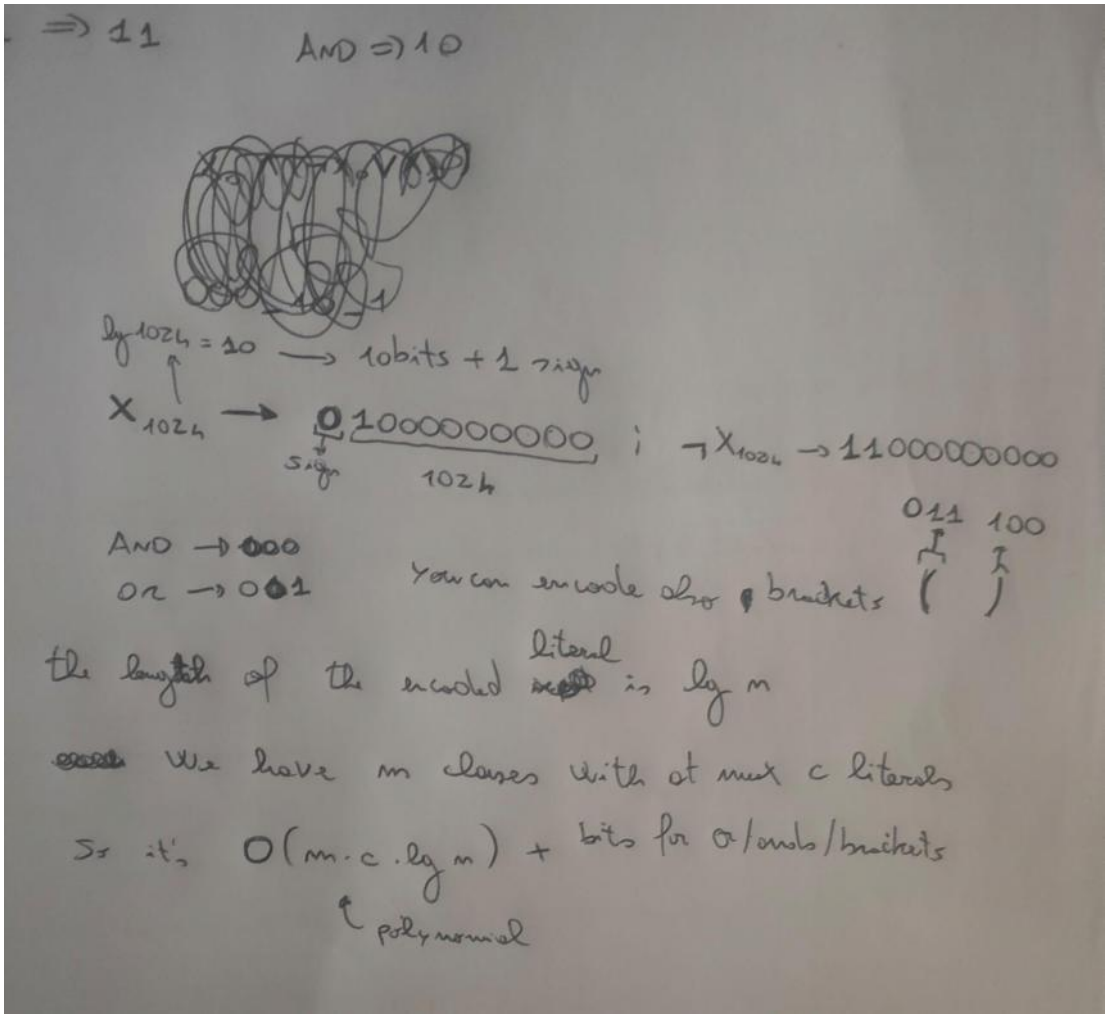
$E = \{(1,2), (2,3), (2,3), (3,3)\}$

$L(V,E)J = L4J \# L2J @ L3J \# L3J \# L3J$

- L4J : L V J
- L2J : SET OF NODES REACH. FROM 1 IN 1 STEP
- L3J : SET OF NODES REACH. FROM 2
- L3J : SET OF NODES REACH FROM 3.

Other usefull encodings

CNF



List encodings

To briefly explain **list encoding**, the sources describe a method that reduces a list to a sequence of **nested pairs** and then transforms that sequence into a strictly binary string [1] [2].

The process generally follows these steps:

- **Recursive Pair Representation**: A list of elements $[b_1, b_2, \ldots, b_n]$ is viewed as a series of nested pairs, such as $((\ldots(b_1, b_2), b_3), \ldots, b_n)$ [1].
- **Use of Separators**: To distinguish between elements within a pair (x, y), a special separator symbol like **#** is used, creating a string such as $x\#y$ [1] [3].
- **Binary Transformation (Bit-Pairing)**: To convert this into a binary string over the alphabet $\{0, 1\}$, a mapping is applied to each symbol to ensure the separator is uniquely identifiable without using a third character:
  - **0** is mapped to **00** [1].
  - **1** is mapped to **11** [1].
  - **#** is mapped to **01** [1].
- **Resulting Length**: The final length of the encoded string $|[b_1, \ldots, b_n]|$ is calculated as the sum of twice the lengths of the individual elements plus twice the number of separators (roughly $\sum_{i=1}^{n} 2|b_i| + 2(n-1)$) [2].

Arrangement with repetition and combinations are very useful to calculate complexity

Type	**Does Order Matter?**	**Repetition Allowed?**	**Formula**
Permutations (using *all* items)	**Yes**	No	$n!$
Arrangements with repetition	**Yes**	Yes	$Choices^{Slots}$
Combinations (using *k* out of *n* items)	**No**	No	$\frac{n!}{k!\,(n-k)!}$